1. Tp sur Spring-security

1.1. A savoir avec spring-boot-starter-security

En Ajoutant seulement ceci dans **pom.xml**

```
<dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-security</artifactId>
</dependency>
```

et redémarrant une application vierge sans aucune configuration.

La console affiche alors un message de ce genre :

Using generated security password: 656a4d96-2f13-46fd-b4b9-2e2c90b3fbb6

This generated password is for development use only. Your security configuration must be updated before running your application in production.

Ce mot de passe (régénéré à chaque démarrage) est pour le username "**user**"

Après avoir saisi "user" et le mot de passe attendu par copier/coller dans la boîte de dialogue , on peut accéder à la page d'accueil principale (index.html) et naviguer vers d'autres pages .

Autrement dit , la sécurité "spring" n'est réellement exploitable qu'avec un minimum de configuration et sans aucune configuration il vaut mieux de pas ajouter spring-boot-starter-security dans pom.xml .

1.2. withoutSecurity

Si en début de développement on souhaite (malgré la dépendance directe ou indirecte spring-boot-starter-security) utiliser la partie web de l'application en mode "sécurité désactivée" on peut utiliser une classe de configuration ressemblant à tp.appliSpring.**WithoutSecurityConfig** avec entre autres `auth.requestMatchers("/**").permitAll()` .

C'est dans ce mode de fonctionnement que les Tps des chapitres précédents ont été effectués.
Avec profils "dev,reInit" sélectionné au démarrage de l'application.

1.3. Sécurité sur Api REST en mode OAuth2/OIDC

- Repérer la dépendance fondamentale ***spring-boot-starter-oauth2-resource-server*** déjà placée dans pom.xml
- Repérer le paramétrage suivant au sein du fichier **application-withSecurity.properties**
  spring.security.oauth2.resourceserver.jwt.issuer-uri=
  https://www.d-defrance.fr/keycloak/realms/sandboxrealm
- Compléter la partie manquante du paramétrage de la sécurité au sein du fichier tp.appliSpring.security.**SecurityConfigForRest**
  A ce stade du Tp , la partie à compléter se situe au niveau de la sous fonction private HttpSecurity ***restFilterChainBuilder***(HttpSecurity http) appelée par le point d'entrée @Bean @Order(1) @Profile("!withoutOAuth2")

public SecurityFilterChain restFilterChainWithOAuth2(HttpSecurity http) .

- Démarrer l'application SpringBoot avec les profils "dev,reInit,**withSecurity**"
- Tester le comportement sécurisé de la partie rest via :
  - une première tentative de compteAjax avec un ajout de compte (label="ccc" , solde=0) menant normalement au message d'erreur 401/Unauthorized
  - une seconde tentative après un login oAuth2/oidc via le compte "admin1/pwd1"

1.4. Sécurité sur Api REST sans OAuth2 avec gestion directe JWT

- Repérer la partie ***passwordEncoder*** présente dans la classe principale *AppliSpringApplication* et son utilisation au sein de la méthode .**create**() de la classe *ServiceClientDirectImpl* .
- Analyser globalement tout le code de la partie t**p.appliSpring.security.generic.standalone** et la dépendance *jjwt-impl* dans pom.xml
- Compléter le code de la méthode **loadUserByUsername** de la classe tp.appliSpring.bank.security.**MyUserDetailsService**
- Analyser la configuration de la partie **restFilterChainWithoutOAuth2()** du fichier tp.appliSpring.security.**SecurityConfigForRest**

- Démarrer l'application SpringBoot avec les profils "dev,reInit,**withSecurity,withoutOAuth2**"
- Tester le comportement sécurisé de la partie rest via :
  - un **standaloneLogin/logout** pour bien repartir de zéro .
  - une première tentative de ***compteAjax*** avec un ajout de compte (label="ccc" , solde=0) menant normalement au message d'erreur 401/Unauthorized
  - une seconde tentative après un login en mode "**standaloneLogin (sans oauth2)**" via le compte "**client_1/pwd**"

1.5. Paramétrage @PreAuthorize sur CompteRestCtrl

- Ajouter sur certaines méthodes de la classe `CompteRestCtrl` des annotations **`@PreAuthorize()`** combinant si besoin `hasRole('ADMIN') or hasRole('CUSTOMER') or hasAuthority('SCOPE_resource.write')` de manière à ce que seuls les administrateurs , "client/customer" ou bien les personnes ayants les scopes suffisants (SCOPE_resource.write ou SCOPE_resource.delete) puisse déclencher les requêtes en mode POST,PUT et DELETE .
- En mode oAuth2 , tester le comportement affiné de la sécurité en tentant les modifications au sein de **compteAjax** après s'être connecté avec le compte **"user1/pwd1" (403/Forbidden)** puis **"admin1/pwd1"**.
- En mode "standaloneLogin (sans oauth2)", tester le comportement affiné de la sécurité en tentant les modifications au sein de **compteAjax** après s'être connecté avec le compte **"user1/pwd1" (403/Forbidden)** puis **"client_1/pwd"**.

1.6. Sécurité sur partie site (thymeleaf) / Tp facultatif

- Analyser la configuration du fichier tp.appliSpring.security.**`SecurityConfigForSite`**
- On voit que le service ***UserDetailsService*** est également utilisé à ce niveau .
- Analyser les parties "**login/logout**" de la classe tp.appliSpring.bank.site.controller.**AppCtrl** ainsi que **advice/ErrorController**
- Analyser src/main/resources/**templates/login.html** et **error.html**
- Tester le comportement sécurisé de la partie "site" via :
  - "site" → "welcome" → "logout" pour bien repartir de zéro .
  - une navigation vers "espace_client" automatiquement reroutée vers "login" où il faut saisir "client_1/pwd" pour aller plus loin
  - un éventuel "logout" pour arrêter la session .

1.7. Test unitaire partiel facultatif de CompteRestCtrl en mode sécurisé

Coder (facultativement) un début de TestCompteRestCtrlWithServiceMockWithOauth2Security en s'inspirant de TestHelloRestCtrlWithMocks.java.txt

1.8. (facultatif) avec projet annexe "mysecurity-autoconfigure"

- Visualiser le contenu du projet annexe "mysecurity-autoconfigure" et la dépendance entre notre projet "debutApplispringWeb" et ce projet optionnel .
- Visualiser le déclenchement automatique de xy.MySecurityConfig via le fichier **org.springframework.boot.autoconfigure.AutoConfiguration.imports** présent dans la partie **src/main/resources** du projet **mysecurity-autoconfigure** (à construire et installer via "*mvn install*")
- Visualiser les paramétrages de type "**mysecurity.area.permit-all-list**" de **application-withSecurity.properties** qui sont spécifiquement analysés par le code interne de mysecurity-autoconfigure
- Faire évoluer le code de la classe `SecurityConfigForRest` en décommentant la partie "//optional config from mysecurity-autoconfigure … myPermissionConfigurer" et en basculant de version de restFilterChainBuilder() .
- Déclencher "mvn clean package" sur le projet "debutAppliSpringWeb"
- Redémarrer l'application en mode sécurisé et tester le bon fonctionnement
- Mémoriser (via copies en commentaires) et modifier les valeurs de "mysecurity.area.permit-all-list" et "mysecurity.area.permit-get-list" de application-withSecurity.properties
- Redémarrer l'application en mode sécurisé et visualiser un comportement altéré
- Restaurer les valeurs appropriées de "mysecurity.area.permit-all-list" et "mysecurity.area.permit-get-list"
- Redémarrer l'application en mode sécurisé et tester le bon fonctionnement